

Funciones en JavaScript

¿Qué es una Función?

Una función es un bloque de código diseñado para ejecutar una tarea específica y puede recibir datos, realizar operaciones sobre ellos y devolver un resultado. Las funciones son una herramienta clave en la programación, ya que permiten organizar el código en secciones lógicas y reutilizables.

¿Por qué Usar Funciones?

1. **1.Modularidad:** Las funciones permiten dividir el código en partes más pequeñas, lo que facilita el manejo de proyectos complejos.
2. **2.Reutilización:** Puedes definir una función una vez y llamarla tantas veces como sea necesario, evitando la duplicación de código.
3. **3.Mantenimiento:** Cuando necesitas realizar cambios, puedes hacerlo en una sola ubicación (la función), lo que reduce el riesgo de errores.

Estructura de una Función en JavaScript

La sintaxis básica para definir una función en JavaScript es:

```
function nombreDeLaFuncion(parametros) {
```

```
    // bloque de código a ejecutar
```

```
}
```

- **nombreDeLaFuncion:** Debe ser descriptivo sobre la tarea que realiza.
- **parametros:** Variables que se pueden usar dentro de la función para recibir información externa. Puedes tener múltiples parámetros o ninguno.
- **bloque de código:** Conjunto de instrucciones que se ejecutan cuando se llama a la función.

Cómo Llamar a una Función

Para ejecutar una función, simplemente **se utiliza su nombre seguido de paréntesis:**

```
nombreDeLaFuncion();
```

Si la función tiene **parámetros**, se deben proporcionar los argumentos **adecuados** dentro de los paréntesis:

```
nombreDeLaFuncion(argumento1, argumento2);
```

Ejemplo:

```
function saludar(nombre) {
```

```
    console.log("Hola, " + nombre + "!");
```

```
}
```

```
saludar("Juan"); // Imprime: Hola, Juan!
```

Parámetros y Argumentos

- **Parámetros:** Los parámetros son como las "instrucciones" que le damos a una función. Son variables que declaramos dentro de los paréntesis de la función y que nos permiten recibir información cuando llamamos a esa función. Estas variables aparecen en la definición de la función. **Ejemplo:** `function miFuncion(parametro1, parametro2).`
- **Argumentos:** Los argumentos son los valores reales que pasamos a la función cuando la llamamos. Esos valores se asignan a los parámetros correspondientes. **Ejemplo:** `miFuncion(valor1, valor2).`

Los argumentos son los valores reales que pasamos a la función cuando la llamamos. Esos valores se asignan a los parámetros correspondientes.

Las funciones pueden tener:

// Sin parámetros:

```
function funcionSinParametros() { ... }
```

// Con un parámetro:

```
function funcionConUnParametro(parametro) { ... }
```

// Con múltiples parámetros:

```
function funcionConMultiplesParametros(param1, param2) { ... }
```

Valores de Retorno

Una función puede devolver un valor usando la palabra clave `return`. Cuando se llama a la función, puede capturar este valor en una variable.

Ejemplo:

```
function sumar(a, b) {
```

```
    return a + b; // Devuelve la suma de a y b
```

```
}
```

```
let resultado = sumar(3, 5);
```

```
console.log(resultado); // Imprime: 8
```

TIP: Si no se especifica un `return`, la función devuelve `undefined` por defecto.

Alcance de Variables

Las funciones también manejan el concepto de **alcance o 'scope'** (en inglés). En pocas palabras, el **alcance/scope determina la visibilidad de las variables** en un programa o script de JavaScript. Hay básicamente dos tipos de alcance:

1. **1.Variables locales:** Declaradas dentro de una función y solo accesibles dentro de esa función.
2. **2.Variables globales:** Declaradas fuera de cualquier función y accesibles en cualquier parte del código.

Ejemplo de alcance:

```
let x = 10; // Variable global
```

```
function mostrarX() {  
  
    let y = 5; // Variable local  
  
    console.log("X es: " + x); // Accede a la variable global  
  
    console.log("Y es: " + y); // Accede a la variable local  
  
}  
  
mostrarX();  
  
console.log(y); // Error: y is not defined
```

Este concepto se mencionará múltiples veces en las próximas lecciones y cursos ya que es un concepto muy importante en **JavaScript**, así que debes entenderlo lo antes posible. Otra manera de entender este concepto podría ser con una analogía como se ilustra a continuación.

Imaginemos que el **scope/alcance** en JavaScript es como un armario donde guardamos nuestras cosas. En este armario, podemos tener diferentes cajones, y cada cajón tiene sus propias cosas.

1. **1.Scope global:** Es como el primer cajón del armario. **Todo lo que guardamos aquí se puede ver y utilizar desde cualquier lugar de nuestro código.** Por ejemplo, si declaramos una variable fuera de cualquier función, se considera global y **puede ser utilizada en cualquier parte del programa.**
2. **2.Scope local:** Ahora, imagina que tenemos un segundo cajón que solo se puede abrir cuando estamos dentro de una habitación especial, como una función. **Todo lo que guardamos aquí solo puede ser utilizado dentro de esa función.** Si intentamos acceder a estas variables **desde fuera, no podremos verlas.** Esto se llama scope local.

3. **3.Funciones y scope:** Cuando definimos una función, creamos un nuevo "cajón" para las variables que están dentro de esa función. **Esto significa que las variables que declares dentro de la función son privadas y no afectan al resto del programa.**

Así que, si tenemos una función que declara una variable llamada x, **solo podemos usar x dentro de esa función**. Si intentamos acceder a x fuera de ella, obtendremos un error porque **no está disponible en el scope global**.

En resumen, **el scope/alcance nos ayuda a organizar nuestro código y a evitar que diferentes partes de nuestro programa se confundan entre sí**. Al trabajar con funciones, el scope **nos permite tener variables que solo se usan en contextos específicos, manteniendo todo más ordenado y evitando conflictos**.

Funciones Anónimas

En JavaScript, puedes definir **funciones sin un nombre**. Estas se llaman funciones anónimas y son útiles en ciertas situaciones, como en **funciones de callback (callback functions en inglés)**.

Ejemplo:

```
const saludar = function(nombre) {  
  
    console.log("Hola, " + nombre + "!");  
  
};
```

```
saludar("María"); // Imprime: Hola, María!
```

Funciones de Flecha

Desde **ES6, JavaScript** introdujo las **funciones de flecha (arrow functions)**, que **son una forma más concisa de escribir funciones**. Estas funciones no tienen su propio contexto de this (no te preocupes si no sabes aún qué es this), lo que puede ser útil en ciertos casos. **Más adelante en tus cursos profundizarás más este tipo de funciones.**

Ejemplo:

```
const multiplicar = (a, b) => a * b;
```

```
console.log(multiplicar(4, 5)); // Imprime: 20
```

Ejemplos de Uso Común de Funciones

1. **1.Cálculos Matemáticos:** Calcular áreas, promedios, etc.
2. **2.Procesamiento de Datos:** Filtrar o transformar datos en arrays.
3. **3.Eventos en el Navegador:** Funciones que responden a eventos, como clics de botones o envío de formularios.

Resumen

Las funciones son una herramienta fundamental en la programación que permiten a los desarrolladores crear código modular, reutilizable y fácil de mantener. Al comprender cómo definir, llamar y manejar parámetros y valores de retorno, los programadores pueden escribir código más eficiente y organizado. El uso de funciones anónimas y de flecha también ofrece flexibilidad en la escritura de funciones, lo que enriquece aún más la capacidad de programación en JavaScript.

Ejercicios clase

Ejercicio 1: Calculadora Simple

Descripción: Crea una calculadora simple que permita realizar operaciones matemáticas básicas: suma, resta, multiplicación y división.

Requisitos:

1. **1.Define cuatro funciones:** sumar, restar, multiplicar, y dividir, cada una de las cuales toma **dos parámetros** y devuelve el resultado de la operación correspondiente.
2. **2.En el programa principal,** llama a cada función con diferentes pares de números y muestra el resultado en la consola.

Ejemplo de implementación:

```
function sumar(a, b) {
```

```
    return a + b;
```

```
}
```

```
function restar(a, b) {
```

```
    return a - b;
```

```
}
```

```
function multiplicar(a, b) {
```

```
    return a * b;
```

```
}
```

```
function dividir(a, b) {
```

```
    if (b === 0) {
```

```
        return "Error: División por cero";
```

```
    }
```

```
    return a / b;
```

```
}
```

// Llamadas a las funciones

```
console.log("Suma: " + sumar(5, 3));
```

```
console.log("Resta: " + restar(5, 3));
```

```
console.log("Multiplicación: " + multiplicar(5, 3));
```

```
console.log("División: " + dividir(5, 0));
```

Ejercicio 2: Conversor de Temperaturas

Descripción: Crea una función que convierta temperaturas entre grados Celsius y Fahrenheit.

Requisitos:

1. Define dos funciones: `celsiusAFahrenheit` y `fahrenheitACelsius`.
2. `celsiusAFahrenheit` debe tomar una temperatura en Celsius y devolver la temperatura equivalente en Fahrenheit.
3. `fahrenheitACelsius` debe tomar una temperatura en Fahrenheit y devolver la temperatura equivalente en Celsius.
4. En el programa principal, llama a ambas funciones con ejemplos de temperaturas y muestra los resultados en la consola.

Ejemplo de implementación:

```
function celsiusAFahrenheit(celsius) {  
  
    return (celsius * 9/5) + 32;  
  
}
```



```
function fahrenheitACelsius(fahrenheit) {
```

```
    return (fahrenheit - 32) * 5/9;
```

```
}
```

```
// Llamadas a las funciones
```

```
console.log("20°C en Fahrenheit es: " + celsiusAFahrenheit(20));
```

```
console.log("68°F en Celsius es: " + fahrenheitACelsius(68));
```